

บทที่ 3

การพัฒนาระบบ

3.1 การพัฒนาส่วนฮาร์ดแวร์

จุดมุ่งหมายหลักในการออกแบบวงจร คือการออกแบบให้วงจรเหล่านี้สามารถส่งค่ากลับมายังคอมพิวเตอร์ และตอบสนองผู้ใช้ได้อย่างถูกต้อง ซึ่งก็จะต้องมีทั้งส่วนอินพุตและเอาต์พุต



รูปที่ 3-1 รูปการติดตั้งอุปกรณ์ที่ได้ออกแบบไว้

จากรูป 3-1 ณ ตำแหน่งหมายเลข 1 และ 2 เป็นส่วนตรวจสอบการบังคับรถจักรยาน และหมายเลข 3 เป็นส่วนสร้างปฏิกิริยาตอบสนอง



รูปที่ 3-2 แสดงรถจักรยานและอุปกรณ์ทั้งหมด

3.1.1 อุปกรณ์ที่ใช้ในการพัฒนาทั้งหมด

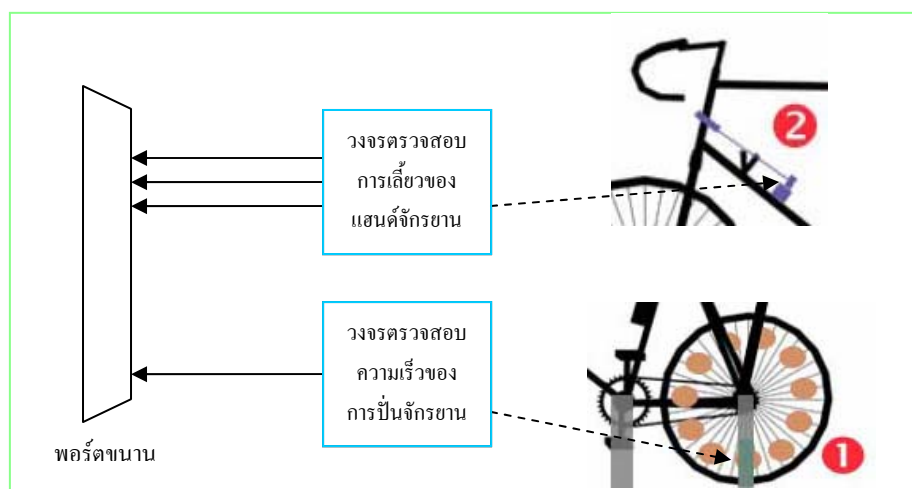
- 1) รถจักรยาน
- 2) ฐานตั้งรถจักรยาน
- 3) ตัวต้านทานปรับค่าได้ (Potentiometer)
- 4) เซ็นเซอร์แสง (Optical Sensor)
- 5) มอเตอร์
- 6) สายยางสำหรับพันรอบแฮนด์จักรยาน
- 7) A/D (Analog to Digital Converter)
- 8) รีเลย์ (Relay)
- 9) บัฟเฟอร์
- 10) ตัวรักษาแรงดัน (Regulator)
- 11) ตัวต้านทาน
- 12) ตัวเก็บประจุ
- 13) สายไฟ
- 14) ปลั๊กไฟ
- 15) หม้อแปลงไฟ 220V – 12V
- 16) สายแพร
- 17) หัว DB-25
- 18) หัวต่อ (Connector)
- 19) บอร์ดไขปลา

การออกแบบวงจรทั้งหมดสามารถแบ่งออกเป็นส่วนหลักๆ ได้ 3 ส่วนด้วยกันคือ

- ส่วนตรวจสอบการบังคับรถจักรยาน
- ส่วนสร้างปฏิกิริยาตอบสนอง
- ส่วนติดต่อระหว่างคอมพิวเตอร์กับฮาร์ดแวร์

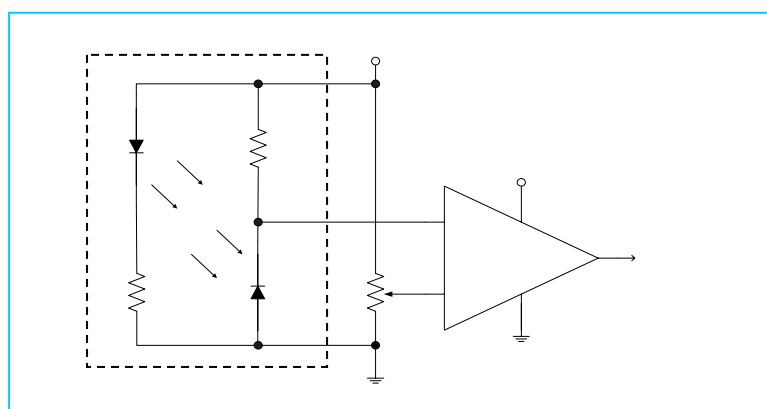
3.1.2 ส่วนตรวจสอบการบังคับรถจักรยาน

ส่วนนี้จะมีการตรวจสอบด้วยกัน 2 จุดคือ ตรวจสอบความเร็วของการปั่นรถจักรยาน และ ตรวจสอบการเลี้ยวของแฮนด์รถจักรยาน



รูปที่ 3-3 โครงสร้างส่วนตรวจสอบการบังคับรถจักรยาน

1) การตรวจสอบความเร็วของการปั่นรถจักรยาน จะใช้เซ็นเซอร์แสงเป็นตัวตรวจสอบ โดยได้ออกแบบวงจรเพื่อที่จะใช้ต่อชุดตรวจสอบความเร็วไว้ดังรูปที่ 3-4

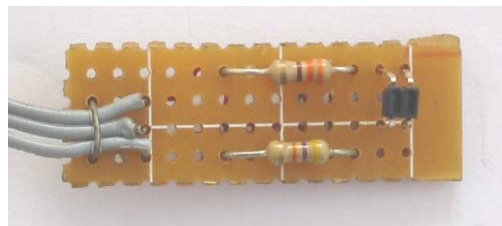
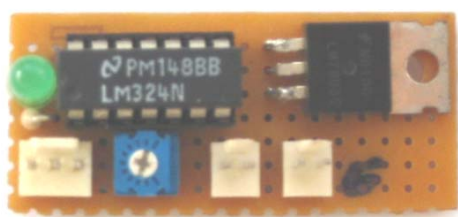


รูปที่ 3-4 วงจรตรวจสอบความเร็วของการปั่นรถจักรยาน

ทำการติดตั้งไมโครคอนโทรลเลอร์ที่มีสีด้านบนของรถจักรยาน เพื่อที่เซ็นเซอร์แสงจะสามารถตรวจสอบการหมุนของล้อได้ดังรูปที่ 3-5 และ 3-6 โดยติดตั้งดังกล่าวกวไว้ทั้งหมด 8 ชิ้นรอบล้อของรถจักรยาน เมื่อล้อหมุน เซ็นเซอร์ที่ตรวจจับอยู่จะส่งค่ากลับมาเป็นความต่างศักย์ไฟฟ้าระหว่าง 0V กับ 5V สลับกันไปเรื่อยๆ โดยความถี่ของค่าความต่างศักย์ไฟฟ้าจะขึ้นกับความเร็วที่เราปั่นรถจักรยาน

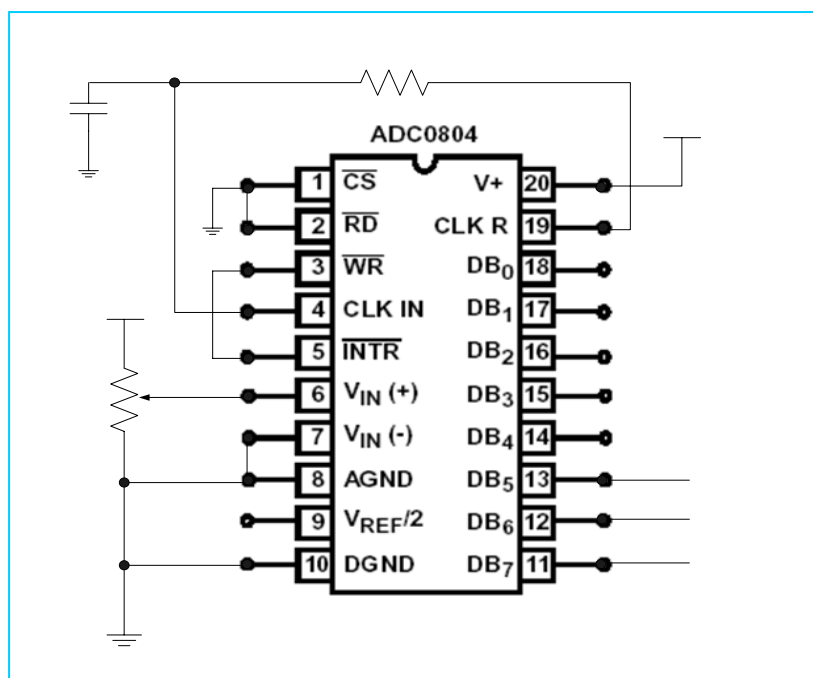


รูปที่ 3-5 กระดาษที่ติดอยู่ที่ซี่โครงล้อ และตัวเซ็นเซอร์



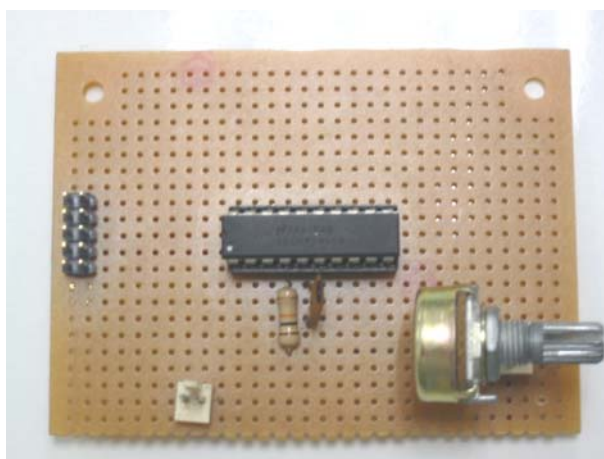
รูปที่ 3-6 วงจรเซ็นเซอร์

2) การตรวจสอบการเลี้ยวของแฮนด์รถจักรยาน เราจะใช้ตัวต้านทานปรับค่าได้เพื่อตรวจสอบระยะการเลี้ยวว่าเลี้ยวไปด้านใด และมุมการเลี้ยวขนาดเท่าไร โดยทำการออกแบบและต่อวงจรดังรูปที่ 3-7 และรูปที่ 3-8 ตามลำดับ ตัวต้านทานปรับค่าได้นั้นจะทำให้ค่าความต่างศักย์เปลี่ยนแปลงแล้วส่งไปให้วงจร A/D แปลงค่าความต้านทานเป็นสัญญาณดิจิทัล 8 บิต



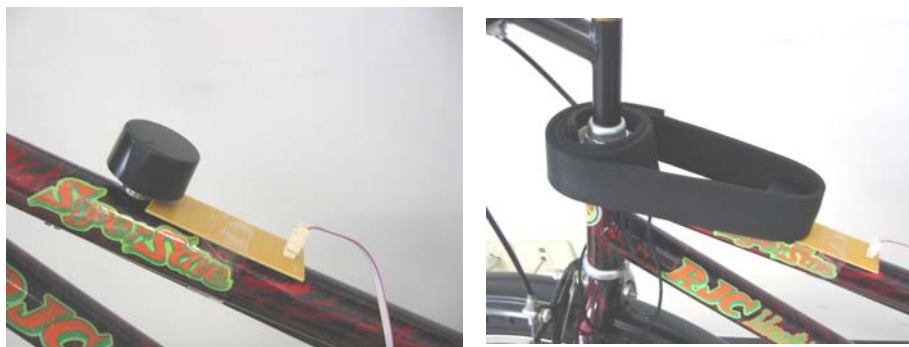
150pF

รูปที่ 3-7 วงจรตรวจสอบการเลี้ยวของเซนส์รถจักรยาน



รูปที่ 3-8 อุปกรณ์ตรวจสอบการเลี้ยว

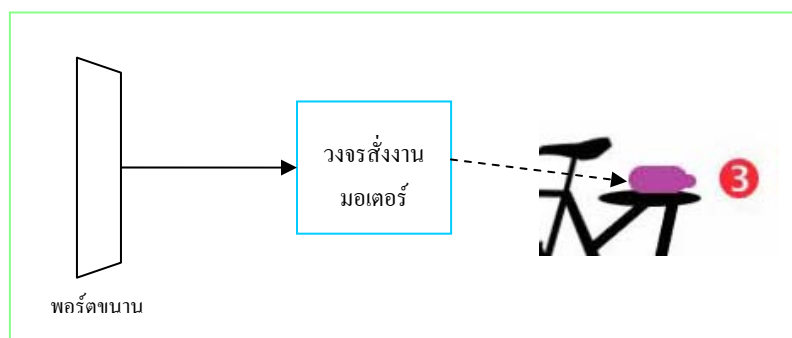
จากนั้นจึงนำตัวต้านทานปรับค่าได้มาคล้องด้วยสายพานซึ่งคล้องอยู่กับคอร์รถจักรยานดังรูปที่ 3-9 เพื่อให้เมื่อเลี้ยวซ้ายหรือเลี้ยวขวาแล้ว ตัวต้านทานปรับค่าได้ก็จะหมุนตามแล้วความต้านทานก็จะเปลี่ยน ซึ่งก็จะทำให้ค่าความต่างศักย์ที่ส่งไปให้วงจร A/D เปลี่ยนไปด้วย



รูปที่ 3-9 ตัวต้านทานปรับค่าได้ที่ติดบนโครงรถจักรยาน และมีสายยางพันอยู่กับโครงจักรยาน

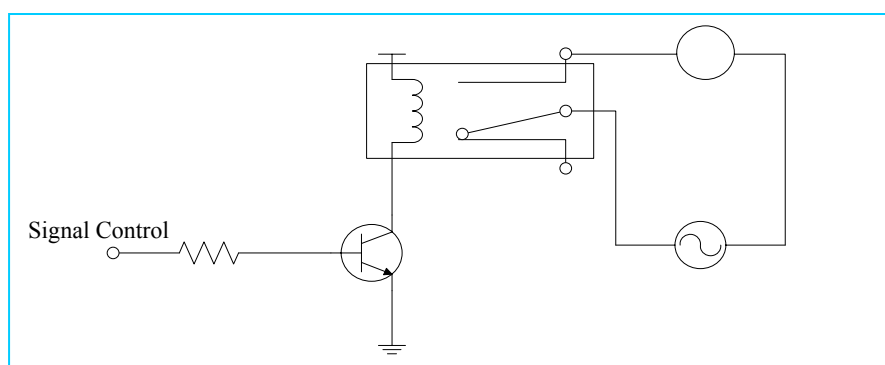
3.1.3 ส่วนสร้างปฏิกิริยาตอบสนอง

ส่วนนี้จะเป็นส่วนที่ตอบสนองต่อผู้ที่ปั่นรถจักรยานด้วยการหมุนของมอเตอร์ที่ติดเป็นลูกเบี้ยว มอเตอร์ดังกล่าวติดตั้งอยู่บนรถจักรยาน โดยจะได้รับคำสั่งให้หมุนที่ส่งมาจากคอมพิวเตอร์ผ่านทางพอร์ตขนาน

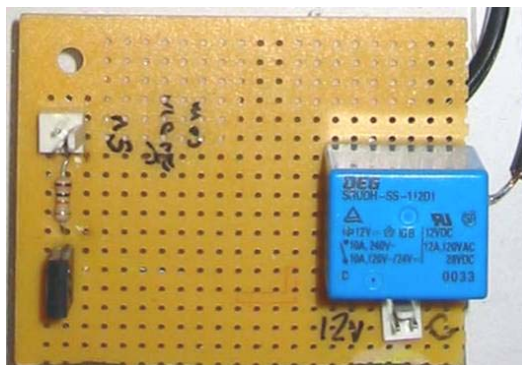


รูปที่ 3-10 โครงสร้างส่วนสร้างปฏิกิริยาตอบสนอง

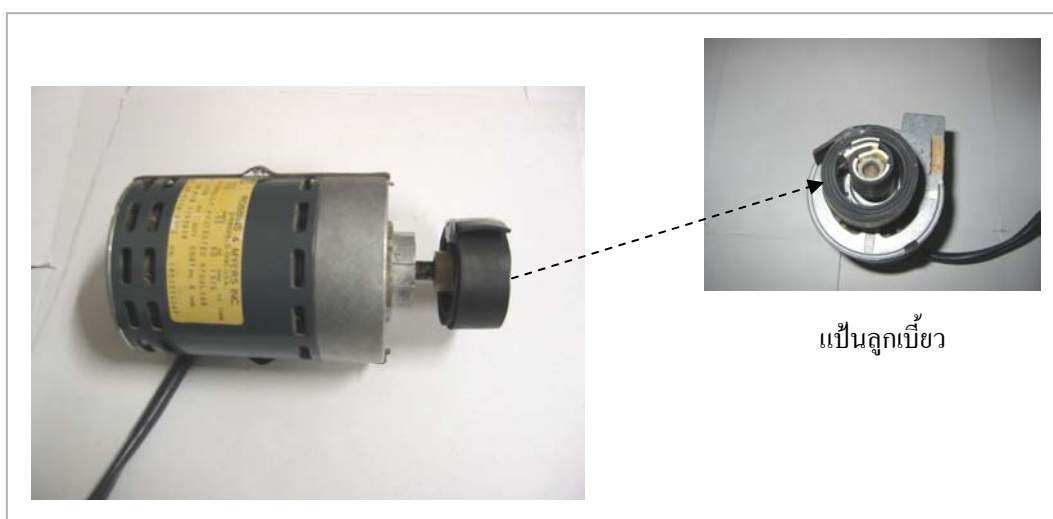
เมื่อเราสั่งให้มอเตอร์หมุน เป็นลูกเบี้ยวจะก่อให้เกิดการสั่นสะเทือน ซึ่งจะทำให้รถจักรยานสั่นไปได้ วงจรควบคุมมอเตอร์ถูกออกแบบไว้ดังรูปที่ 3-11 โดยมอเตอร์มีรีเลย์เป็นตัวควบคุมการจ่ายกระแส และภาพอุปกรณ์ทั้งหมดแสดงไว้ดังรูปที่ 3-12



รูปที่ 3-11 วงจรควบคุมมอเตอร์



รูปที่ 3-12 อุปกรณ์ควบคุมมอเตอร์

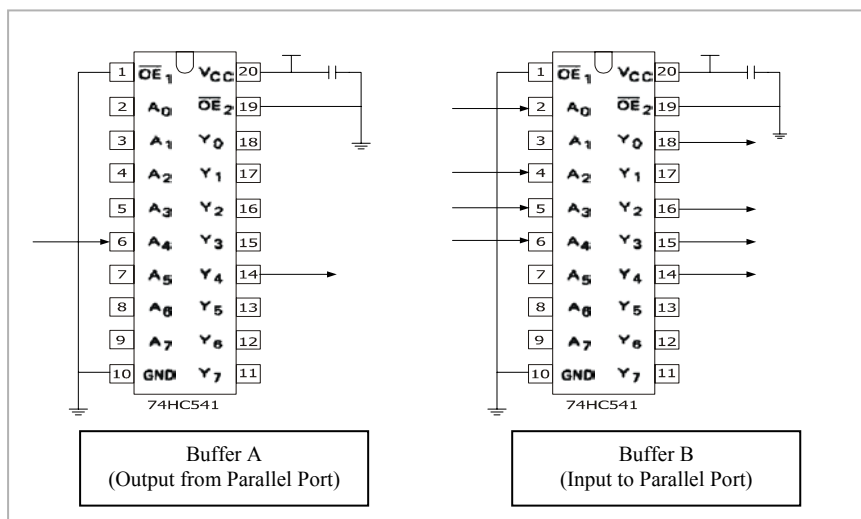


รูปที่ 3-13 มอเตอร์ไฟฟ้า

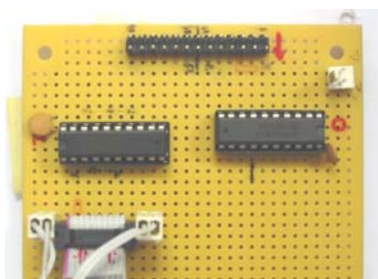
เมื่อเราทำการส่งสัญญาณจากคอมพิวเตอร์ที่มีสถานะเป็น 1 จะทำให้มอเตอร์หมุนซึ่งก็จะทำให้รถจักรยานยนต์ไปด้วย และเมื่อต้องการที่จะหยุดส่งก็ส่งค่าสถานะ 0 มาเพื่อสั่งให้มอเตอร์หยุดการทำงาน การที่เมื่อมอเตอร์หมุนแล้วทำให้รถจักรยานยนต์นั้น เกิดขึ้นจากการติดตั้งเป็นลูกเบี้ยวไว้ที่แกนของมอเตอร์ และเมื่อมอเตอร์หมุนก็จะทำให้เป็นลูกเบี้ยวหมุน ซึ่งจะเหวี่ยงไปมาทำให้เกิดการสั่นได้

3.1.4 ส่วนติดต่อระหว่างคอมพิวเตอร์กับฮาร์ดแวร์

ส่วนนี้จะกล่าวถึงการติดต่อระหว่างอุปกรณ์ต่างกับพอร์ตขนาน ว่าติดต่อกันผ่านพอร์ตใด และ บิตใดบ้าง แต่เนื่องจากการที่จะติดต่อกับพอร์ตขนานโดยตรงนั้นเป็นการเสี่ยงที่อาจจะเกิดอันตรายต่อพอร์ตขนานที่เกิดจากความบกพร่องของวงจรได้ จึงได้เพิ่มส่วนของไอซีบัฟเฟอร์เข้าไปแทรกระหว่างอุปกรณ์ต่างๆกับพอร์ตขนาน ซึ่งได้ออกแบบวงจรไว้ดังรูปที่ 3-14 และเมื่อต่ออุปกรณ์เสร็จจะเป็นดังรูปที่ 3-15



รูปที่ 3-14 วงจรบัฟเฟอร์



รูปที่ 3-15 อุปกรณ์บัฟเฟอร์

จากรูป 3-14 ทางด้านซ้ายของวงจรนั้นจะเป็นส่วนที่ทำการส่งค่าไปควบคุมมอเตอร์ให้ทำงานจากคอมพิวเตอร์ผ่านทางพอร์ตขนาน ซึ่งก็คือส่งไปยังขาที่ 6 และส่งต่อไปจากขาที่ 14 และทางด้านขวานั้นจะเป็นส่วนรับค่ามาจากอุปกรณ์ต่างๆ เพื่อส่งต่อไปยังพอร์ตขนานแล้วส่งเข้าสู่คอมพิวเตอร์ โดยขาที่ 2 จะรับค่ามาจากส่วนตรวจสอบการปั่นรถจักรยาน และขาที่ 4-6 จะรับค่ามาจากส่วนตรวจสอบการเลี้ยวซ้ายเลี้ยวขวา

โดยการติดต่อกับพอร์ตขนานนั้นในพอร์ตที่ใช้รับข้อมูลมาจากส่วนตรวจสอบการบังคับรถจักรยาน นั้นได้เลือกใช้พอร์ตแสดงสถานะ (379h) เนื่องจากเป็นพอร์ตที่ใช้สำหรับรับข้อมูลอยู่แล้ว โดยไม่ต้องมีการส่งค่าอะไรไปเพื่อรับข้อมูลเลย

และสำหรับส่วนที่ใช้ควบคุมการทำงานของมอเตอร์นั้นได้เลือกใช้พอร์ตค่า (378h) เนื่องจากพอร์ตค่าเป็นพอร์ตที่มีไว้สำหรับส่งข้อมูลออกไปอยู่แล้ว จึงทำให้สามารถส่งค่าที่ออกไปได้เลยโดยไม่ต้องตั้งค่าหรือต้องเปลี่ยนแปลงค่าใหม่ก่อนส่ง

โดยสามารถระบุที่ใช้ในแต่ละพอร์ตสำหรับการติดต่อระหว่างฮาร์ดแวร์และคอมพิวเตอร์ผ่านทางพอร์ตขนานได้ดังตารางที่ 3-1

Signal Name	Register bit	Signal Pin	Buffer Pin	ลักษณะการใช้งาน
Data bit 4	D4	6	6A	ส่งสัญญาณไปควบคุมมอเตอร์
Select	S4	13	18B	รับสัญญาณมาจากส่วนตรวจสอบการปั่นรถจักรยาน
PaperEnd	S5	12	16B	S5, S6 และ S7 รับค่ามาจากส่วนตรวจสอบการเลี้ยง
nAck	S6	11	14B	
Busy	S7	10	15B	

(D = Data port, S = Status port, A = BufferA (output from parallel port), B = BufferB (input to parallel port))

ตารางที่ 3-1 แสดงการเชื่อมระหว่างบิตของพอร์ตขนาน กับส่วนต่างๆของอุปกรณ์ภายนอก

3.2 การพัฒนาด้านซอฟต์แวร์

3.2.1 วิธีการโหลด .x file เข้ามาในโปรแกรม

หลังจากที่ได้ออกแบบและสร้างโลกและวัตถุ 3 มิติจากโปรแกรม 3D Studio Max (3DsMax) แล้วนั้น เราจำเป็นต้อง export ออกเป็น .x file ซึ่งสำหรับ 3DsMax นั้นเราต้องใช้โปรแกรมเสริม (Plug-in) ในที่นี้เราใช้ตัวที่มาพร้อมกับ DirectX SDK Extra ขั้นตอนต่อไปคือการโหลดโมเดลเข้ามาในโปรแกรม DirectX ของเรา

ในโปรแกรมเราได้สร้างคลาส CMesh ขึ้นมาสำหรับการโหลดและเรนเดอร์ mesh จาก .x files โดยที่ตัว CMesh Constructor จะเอาไว้โหลด mesh จาก .x file และเก็บไว้ในหน่วยความจำ ตัว Destructor เอาไว้คืนหน่วยความจำที่เราเอาไว้เก็บ mesh และ เมฆอด render แสดงถึงวิธีการ เรนเดอร์ ตัว mesh ที่ได้โหลดเอาไว้

CMesh Constructor รับค่า 2 ตัวแปร ตัวแรกเป็นพอยน์เตอร์ชี้ไปที่ ตัว device และตัวที่ 2 เป็นสตริงที่เอาไว้เก็บพาทของ .x file ที่จะเอาไว้โหลด เราใช้ฟังก์ชัน D3DXLoadMeshFromX ในการโหลด Mesh เข้าไว้ในหน่วยความจำ จากนั้นเราได้สร้าง array ขึ้นมา 2 ตัว ตัวแรกเอาไว้เก็บ material อีกตัวเอาไว้เก็บ texture ของโมเดล

ตัวอย่างของการ โหลดโมเดล .x file

```
m_pMeshWorld = new CMesh(m_pD3DDevice, "myworld2.x")
```

3.2.2 การรับข้อมูลจากตัวรถจักรยาน

เนื่องจากเราได้พัฒนาโปรแกรมบนระบบปฏิบัติการ Windows2000 และ Windows XP จึงต้องมีการใช้ไครเวอร์เพื่อการมีสิทธิ์เรียกใช้พอร์ตนานได้อย่างเต็มที่ ในที่นี้เราใช้โปรแกรม PrintIO เป็นไครเวอร์เพื่อการติดต่อ และหลังจากที่ได้เริ่ม ไครเวอร์ แล้วโปรแกรมของเราก็จะพร้อมที่จะรับส่งค่าข้อมูลผ่านทางพอร์ตนาน

สำหรับการอ่านค่าข้อมูล มีด้วยกัน 2 ข้อมูลคือ ข้อมูลการเลี้ยวซ้ายขวา และ ข้อมูลการปั่นรถจักรยาน

เนื่องจากค่าที่อ่านได้จากการใช้คำสั่ง `_inp(0x379)` จะได้บิตที่ไม่ต้องการมาด้วย คือบิต S2,S1,S0 ดังนั้นในการอ่านค่าแต่ละครั้งจะทำให้ได้ค่าที่ไม่ถูกต้องตามความเป็นจริง เราจึงทำโอเปอร์เรชัน and ด้วยค่า 248 ซึ่งจะได้โค้ดดังนี้

```
temp = (_inp(0x379) & 248 );
```

แต่เนื่องจากว่าบิต S7 นั้นถูกกลับสถานะอยู่เราจึงต้องทำการ XOR กับ 128 อีกครั้งซึ่งจะได้โค้ดดังนี้

```
temp = (_inp(0x379) & 248 ^ 128);
```

แต่เพื่อให้สะดวกเราจึงแยกค่าจากข้อมูลการเลี้ยวซ้ายขวา (S7,S6,S5) เก็บไว้ในตัวแปร RVary

```
Rvary = (_inp(0x379) & 248 ^ 128) & 224;
```

และค่าจากข้อมูลการปั่นรถจักรยาน (S4) เก็บไว้ในตัวแปร Sensor

```
Sensor = (_inp(0x379) & 248 ^ 128) & 16;
```

3.2.3 การคำนวณทิศทางเคลื่อนที่

ลักษณะของมุมมองบุคคลที่ 1 หรือมุมมองของผู้เล่นในเกมแนว First-Person Shooting นั้นจะให้ภาพเหมือนกับการมองเห็นจริงของผู้เล่น ซึ่งในโปรแกรมของเรา ได้ใช้มุมมองนี้เป็นมุมมองในการเลี้ยวซ้ายขวา และใช้มุมมองของกล้องอิสระในการเคลื่อนที่ไปมาภายในโลก 3 มิติ ลักษณะของมุมมองแบบอิสระ ก็คือ ไม่ว่าเราจะมองไปทางไหน หรือองศาไหน ถ้าเราเคลื่อนที่ไปข้างหน้า มันจะต้องเคลื่อนที่ตรงเข้าหาจุดที่เรามองได้อย่างแม่นยำ หรือถ้าเราถอยหลัง ก็ต้องยังมีสิ่งที่เรามองอยู่ข้างหน้าเสมอ แต่เราจะค่อยๆ ถอยห่างออกจากจุดนั้น ดังนั้น ถ้าเราต้องการขึ้นลงสะพาน เราจะต้องคำนึงถึงแกน y เพิ่มเข้ามาด้วย

1) การเลี้ยวซ้ายขวา

สำหรับการเลี้ยวซ้ายขวานั้น ก็คือการหมุนกล้องซ้ายขวา ถ้าเป็นคำในภาษาอังกฤษจะเรียกการหมุนแบบนี้ว่า Yaw การหมุนกล้องซ้ายขวานี้ ที่จริงแล้วก็คือการหมุนรอบแกน y นั่นเอง โดยตอนเริ่มต้นเราต้องเก็บค่ามุมรอบแกน y ไว้ก่อนเป็นตัวตั้งต้นจะใช้ตัวแปรที่ชื่อว่า CenterAngle จากนั้นเมื่อมีการเลี้ยวซ้ายขวาก็ให้มีการบวกหรือลดค่าจาก CenterAngle โดยการบวกหรือลดจะขึ้นอยู่กับค่า RVary แล้วเก็บเป็นมุมรอบแกน y เอาไว้จะใช้ตัวแปรที่ชื่อว่า g_fAngleY ปกติแล้ว ค่าของมุมรอบแกน y จะหมุนทวนเข็มนาฬิกา โดยมุม 0.0 เรเดียนหรือ 0.0 องศา จะหันไปทางแกน +z แล้วมุมจะเพิ่มขึ้นไปทางแกน -x, -z, +x

แล้วกลับมาครอบที่ +z ตามลำดับ ดังนั้นถ้ามีการบังคับแฮนด์รถจักรยานไปทางซ้าย ก็จะเป็นการเพิ่มค่ามุม ถ้าบังคับไปทางขวาก็จะเป็นการลดค่ามุม เช่น

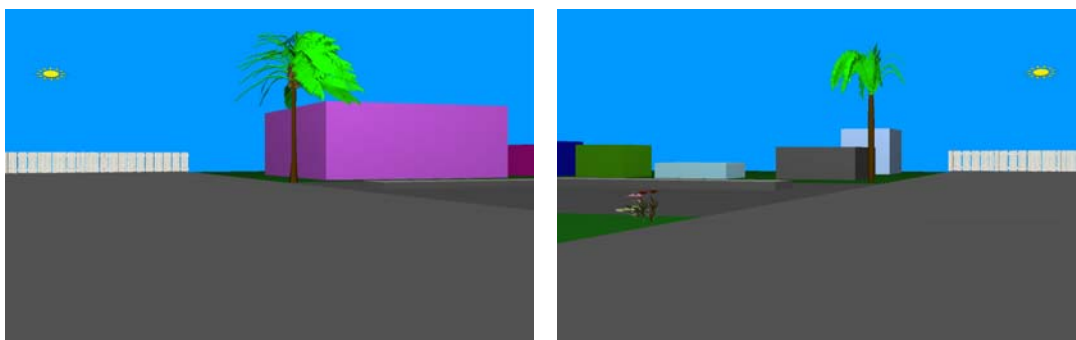
```
if(Rvary == 32)
{
    g_fAngleY = CenterAngle + 2*ROTATION_SPEED;
}
```

จากนั้นในขั้นตอนการสร้าง View Matrix เราจะใช้ฟังก์ชัน D3DXMatrixRotationY() เพื่อสร้างเมทริกซ์ ผลลัพธ์ที่ได้จากค่าของมุมรอบแกน y

```
D3DXMatrixRotationY(&matRotationY, D3DXToRadian(g_fAngleY));
```



รูปที่ 3-16 ตัวอย่างรูปมุมมองบุคคลที่ 1



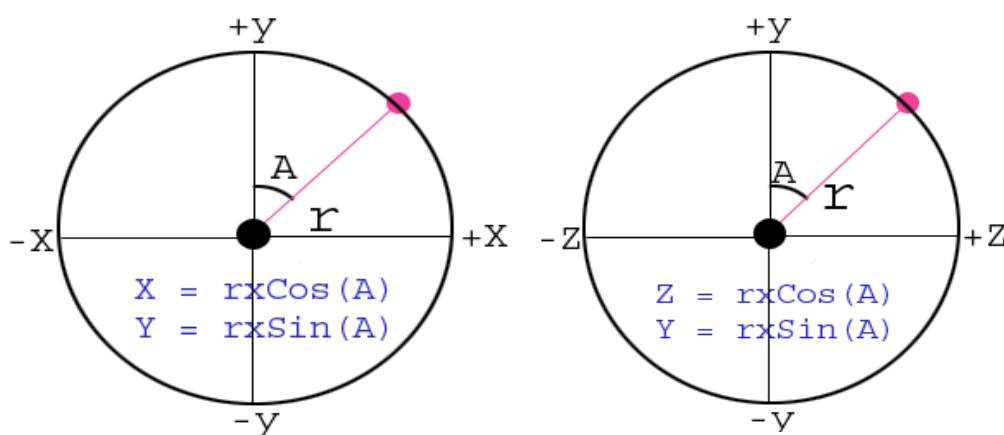
รูปที่ 3-17 แสดงถึงมุมมองเมื่อมีการเลี้ยวขวา และซ้าย

2) การเดินหน้าและถอยหลัง

สำหรับการเคลื่อนที่ไปมาภายในโลก 3 มิติ ก็คือการเคลื่อนที่ในแนวระนาบของกล้อง ซึ่งก็คือการเคลื่อนที่ที่อยู่ในแกน x และ z ของ Worldspace (เพราะแกนทั้งสองนี้มันตั้งฉากกันอยู่ในแนวระนาบ) โดยแกน +z จะพุ่งตรงไปข้างหน้าเข้าหาจอ, แกน -z พุ่งไปทางด้านหลัง, แกน -x อยู่ทางซ้าย และแกน +x อยู่ทางด้านขวา เราอาจจะคิดว่า การเคลื่อนที่ไปข้างหน้าและหลังก็คือการเพิ่มและลดค่า z ส่วนการเคลื่อนที่ไปทางซ้ายและขวาก็คือการเพิ่มและลดค่า x ความคิดนี้ถูกต้องในกรณีที่เราหันหน้าเข้าหาแกน +z ของ Worldspace ตลอดเวลาเท่านั้น แต่อย่าลืมว่า เรามีการหมุนตัวด้วย ดังนั้น แกนที่เราหันหน้าเข้าหาอาจจะอยู่ระหว่างแกน x กับ z ของ Worldspace ก็ได้ ซึ่งจะต้องมีการเพิ่มและลดค่าในแกนทั้งสองพร้อมๆ กันตามมุมที่หมุน

และเนื่องจากแกน y ตั้งฉากกับแกน x และ z ดังนั้นเราจะต้องพิจารณาแกน y ถึง 2 ครั้ง ครั้งแรกพิจารณาในกรณีของแกน y กับแกน x ตามรูป 3-18 ก เป็นภาพใน Worldspace เมื่อเรามองอยู่ในแนวแกน z เข้าไป (มองเข้าไปในแนวแกน +z) จะได้ว่า มุม A คือมุมรอบแกน z ดังนั้น ค่าของตำแหน่งกล้องในแกน y จะมีค่าเท่ากับ รัศมีคูณด้วยค่า $\sin(\text{มุมรอบแกน } z)$ และค่าของตำแหน่งกล้องในแกน x เท่ากับ รัศมีคูณด้วยค่า $\cos(\text{มุมรอบแกน } z)$

ทีนี้เราจะพิจารณาในกรณีของแกน y กับแกน z กันบ้าง ตามรูปที่ 3-18 ข แต่เปลี่ยนจากแกน x เป็น z โดยเราจะมองเข้าไปในแนวแกน -x จะได้ว่า มุม A ก็คือมุมรอบแกน x และค่าของตำแหน่งกล้องในแกน y จะยังคงเท่าเดิม (เพราะแกน x และ z มันใช้แกน y ร่วมกัน) ส่วนค่าของตำแหน่งกล้องในแกน z จะเท่ากับ รัศมีคูณด้วย $\cos(\text{มุมรอบแกน } x)$



(ก) มุมรอบแกน y กรณีของแกน x กับ z

(ข) มุมรอบแกน y กรณีของแกน y กับ z

รูปที่ 3-18 การคำนวณค่าตำแหน่งของกล้อง

เนื่องจากแกน x กับแกน z มันอยู่ในแนวระนาบเดียวกัน ดังนั้น มุมรอบแกน x กับ มุมรอบแกน z ก็จะมีค่าเท่ากัน ซึ่งจะใช้อันไหนก็ได้ แต่ในโปรแกรมจะใช้มุมรอบแกน x ซึ่งใช้ชื่อตัวแปรว่า `g_fAngleX` สำหรับรัศมินั้นก็คือระยะทางที่กล้องเคลื่อนไป หรือเป็นความเร็วในการเคลื่อนที่ก็ได้ เพราะถ้าระยะห่างมากก็จะเคลื่อนที่ได้เร็ว แทนด้วยค่าคงที่ชื่อ `MOVING_SPEED` และให้ตัวแปร `g_fPosX`, `g_fPosZ` และ `g_fPosY` แทนตำแหน่งของกล้องในแนวแกน x, z และ y ตามลำดับ

ฟังก์ชัน `sinf()` และ `cosf()` จะส่งค่ากลับมาเป็นจำนวนจริง และจะต้องใช้มุมแบบเรเดียนเท่านั้น จึงต้องแปลงจากองศาเป็นเรเดียนโดยใช้ฟังก์ชัน `D3DXToRadian()` นอกจากนี้ จริงๆ แล้วการเคลื่อนที่ไปข้างหน้าก็คือการบวกค่า `x` และ `z` แต่อย่าลืมว่า ตอนที่เรากำหนดแกนจากรูปการหาจุดบนเส้นรอบวง ให้มาเป็นแบบ `Worldspace` นั้น แกน `x` (หรือแกน `y` เดิม) มันสลับค่าบวกและลบกันอยู่ ดังนั้นแทนที่จะบวกค่า `x` (คือแทนที่จะใช้ `+=`) ก็ต้องเป็นลบแทน (ใช้ `-=`)

จะได้สูตรการเคลื่อนที่ดังต่อไปนี้

```
- g_fPosX -= sinf(D3DXToRadian(g_fAngleY)) * cosf(D3DXToRadian(g_fAngleX)) *  
MOVING_SPEED  
- g_fPosY += sinf(D3DXToRadian(g_fAngleX)) * MOVING_SPEED;  
- g_fPosZ += cosf(D3DXToRadian(g_fAngleY)) * cosf(D3DXToRadian(g_fAngleX)) *  
MOVING_SPEED;
```

จะเห็นว่า เป็นการคูณค่า `cos` ของมุมรอบแกน `x` เข้าไปที่ตำแหน่งในแนวแกน `x` และ `z` ส่วนค่าของตำแหน่งของกล้องในแนวแกน `y` จะเป็นค่า `sin` ของมุมรอบแกน `x` ซึ่งใช้ค่า `sin` โดดๆ (เพราะแกน `x` และ `z` มันใช้แกน `y` ร่วมกัน) และไม่ต้องเอาค่า `cos` มาคูณ (เพราะจากรูป ค่า `cos` มันเป็นค่าในแนวระนาบของแกน `x` และ `z` ไม่ใช่ของแกน `y`)

สำหรับการเคลื่อนที่ถอยหลัง ก็แค่กลับเครื่องหมายบวกเป็นลบ ลบเป็นบวก ดังนี้

```
- g_fPosX += sinf(D3DXToRadian(g_fAngleY)) * cosf(D3DXToRadian(g_fAngleX)) *_  
MOVING_SPEED;  
- g_fPosY -= sinf(D3DXToRadian(g_fAngleX)) * MOVING_SPEED;  
- g_fPosZ -= cosf(D3DXToRadian(g_fAngleY)) * cosf(D3DXToRadian(g_fAngleX)) *_  
MOVING_SPEED;
```

อีกสิ่งหนึ่งที่ต้องพิจารณาคือ เมื่อมีการเลี้ยวซ้ายขวาและการเคลื่อนที่ ค่ามุมรอบแกน y จะต้องเปลี่ยนไปด้วยพร้อมทั้งค่ามุม CenterAngle ก็จะต้องเปลี่ยนไปด้วยขนาดที่เท่ากันด้วย เพราะเมื่อเราเลี้ยวกลับให้ตรงเหมือนเดิมแล้ว มุมที่เราเห็นก็จะเปลี่ยนไปจากเดิม เช่น

```
if(Rvary == 32)
{
    g_fAngleY = CenterAngle + 2*ROTATION_SPEED;
    CenterAngle = CenterAngle + 2*ROTATION_SPEED;
}
```

เมื่อได้ค่าตำแหน่งของกล้องในแนวแกน x, y และ z แล้ว ดังนั้นเราจึงคำนวณ View Matrix ของกล้องได้ตามลำดับดังนี้

```
vecEyePoint = D3DXVECTOR3(g_fPosX, g_fPosY, g_fPosZ);
vecLookatPoint = D3DXVECTOR3(g_fPosX, g_fPosY, g_fPosZ + 0.1f);
vecUp = D3DXVECTOR3(0.0f, 1.0f, 0.0f);
D3DXMatrixLookAtLH(&matView, &vecEyePoint, &vecLookatPoint, &vecUp);
```

จากนั้นต้องนำไปรวมกับ เมทริกซ์ ที่เกิดจากการหมุนรอบแกน x และ y ที่คำนวณได้ก่อนหน้านี้ ก็จะได้อุมมองการเคลื่อนที่ตามที่ต้องการ

```
D3DXMatrixMultiply(&matView, &matView, &matRotation);
```

3.2.4 การตรวจสอบการขึ้นลงสะพาน



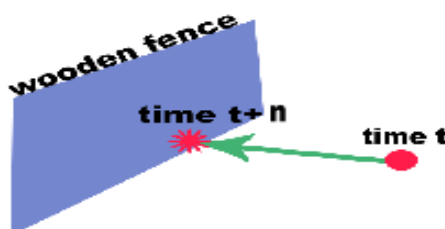
รูปที่ 3-19 การกำหนดความลาดชันของสะพาน

การขึ้นลงสะพานนั้นจะต้องทำการคำนวณหาตำแหน่งของสะพานก่อนว่าอยู่ตรงจุดไหนและขนาดเท่าไร แล้วทำการแบ่งสะพานออกเป็น 7 ส่วนด้วยกันตามแถบสีในรูปที่ 3-19 โดยเราจะแบ่งจากตำแหน่งและขนาดที่ได้มา เพื่อกำหนดให้เมื่อเวลาเราเคลื่อนที่ผ่านแต่ละส่วนให้เงยหน้าขึ้นหรือก้มหน้าลง โดยในส่วนสีส้มนั้นจะเป็นส่วนที่เงยหน้าขึ้นเรื่อยๆ ส่วนสีฟ้านั้นจะก้มหน้าลงเรื่อยๆ ส่วนสีเทาตรงกลางจะให้มองเห็นนานกับพื้น และในส่วนที่ไม่ได้ใส่สีไว้หมายความว่าไม่ให้เปลี่ยนแปลงมุมมอง

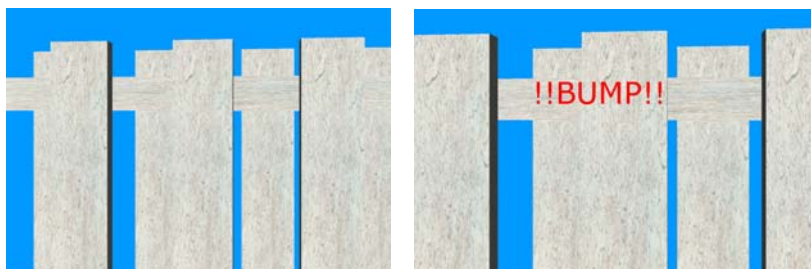
3.2.5 การตรวจสอบการชน

เราจะมีตรวจสอบการชนตลอดเวลาว่า ตำแหน่งของรถจักรยานปัจจุบันอยู่ในบริเวณของวัตถุหรือบริเวณที่ถูกกำหนดไว้ว่าเข้าไปไม่ได้หรือไม่ เช่น รั้ว กำแพง หรือขอบสะพาน เป็นต้น โดยใช้เมธอด CheckCollision()

ตัวอย่าง จากรูปที่ 3-20 ณ เวลา t รถจักรยานอยู่ที่ตำแหน่งวงกลมสีแดง เมื่อเคลื่อนที่เข้าไปหากำแพง ณ เวลา $t+n$ ตำแหน่งของรถจักรยานจะอยู่ในบริเวณของตัวรั้วทำให้เกิดการชนขึ้นที่รั้ว ถ้าเกิดการชนขึ้นจะทำให้ไม่สามารถเดินต่อไปข้างหน้าจนกว่าจะหันไปทางที่ไม่ชนก่อน และจะปรากฏข้อความว่า “!!BUMP!!” ขึ้นมาบนจอภาพ ดังรูปที่ 3-21



รูปที่ 3-20 วิธีตรวจสอบการชน

ภาพก่อนชน ณ เวลา t ภาพขณะชน ณ เวลา $t+n$

รูปที่ 3-21 แสดงภาพก่อนชนและขณะชนรั้ว

จากนั้นเราจะส่งค่าออกไปสั่งมอเตอร์ที่ติดอยู่กับรถจักรยานให้ลั่น โดยจะส่งงานผ่านพอร์ตดาต้า (378h) ซึ่งจะแบ่งเป็น 2 คำสั่ง คือ

สั่งให้มอเตอร์ทำงาน

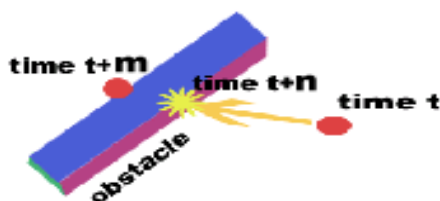
```
_outp(0x378,0xff);
```

สั่งให้มอเตอร์หยุดทำงาน

```
_outp(0x378,0);
```

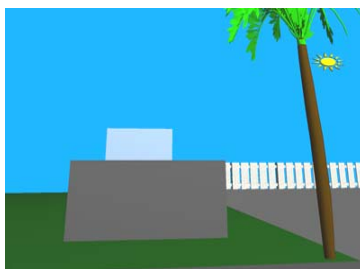
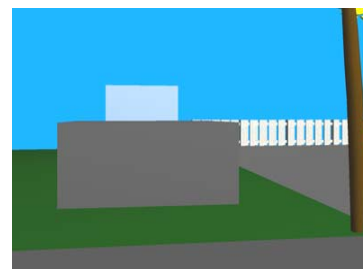
3.2.6 การตรวจสอบการเคลื่อนที่ผ่านเกาะกลางถนน

การเคลื่อนที่ผ่านเกาะกลางถนนนั้นเราได้มีการกำหนดตำแหน่งของเกาะกลางถนนไว้แล้วจากโลกเสมือนจริงที่เราสร้าง และเมื่อเราเคลื่อนที่ผ่านตำแหน่งที่เป็นเกาะกลางถนน เราก็จะกำหนดให้ตำแหน่งที่มองลอยจากพื้นขึ้นไปอีกระดับหนึ่ง เพื่อให้เกิดความรู้สึกเหมือนกับรถจักรยานกระเด็นขึ้นจากพื้นไปอยู่บนเกาะกลางถนน



รูปที่ 3-22 การพิจารณาดำเน่งรถจักรยานเมื่อผ่านเกาะกลางถนน

จากรูปที่ 3-22 ณ เวลา t เมื่อรถจักรยานอยู่ที่ตำแหน่งวงกลมสีแดง เมื่อเคลื่อนที่ถึงเกาะกลางถนน ณ เวลา $t+n$ ตำแหน่งของมุมมองจะทำการยกขึ้นจนกว่ารถจักรยานจะเคลื่อนที่ผ่านตำแหน่งวงกลมสีแดง ณ เวลา $t+m$ จึงค่อยให้ตำแหน่งของมุมมองขกกลับมาที่เดิม

ณ เวลา t ก่อนผ่านเกาะกลางณ เวลา $t+n$ ขณะผ่านเกาะกลางถนนณ เวลา $t+m$ หลังผ่านเกาะกลางถนน

รูปที่ 3-23 แสดงให้เห็นถึงมุมมองขณะเคลื่อนที่ผ่านเกาะกลางถนน